CIS I · ASSIGNMENT 2 · CODE WALKTHROUGH

How the calibration program is built

Architecture, data flow, and what every file and function does

THE PROGRAM IN ONE PARAGRAPH

The course gives you a **simulator** (a fake robot + fake tracker that behave like the real ones, including their hidden errors) and four empty "stub" modules in `src/student/`. Our code fills those stubs. A **notebook** drives everything in the order of the handout: design the scene → home the robot → plan and collect calibration poses → fit an improved kinematic model (运动学模型) → evaluate it against the simulator's ground truth. A **test suite** checks every building block, and a few **report scripts** turn the notebook's numbers into the written report.

Three layers (三层结构)

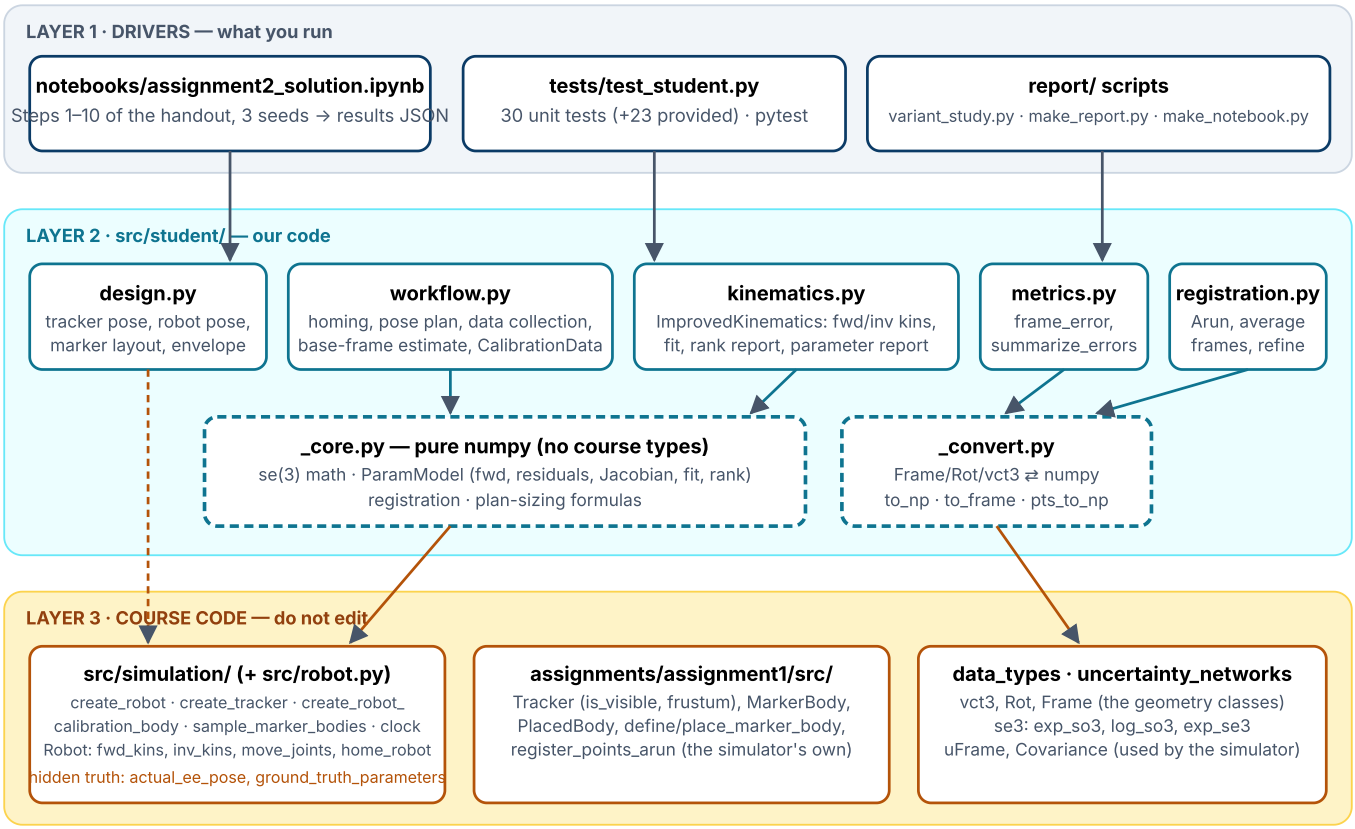


Figure A. The three layers. Arrows point from caller to callee. The dashed boxes hold no course types: `_core.py` works on plain 4×4 numpy arrays and `_convert.py` translates at the boundary.

WHY THE EXTRA LAYER (_CORE / _CONVERT)?

The course's `Frame` / `Rot` / `vct3` classes are convenient but slow and hard to test outside the course repo. All of the real math (forward kinematics 正运动学, residuals, Jacobian 雅可比矩阵, least squares 最小二乘) lives in `_core.py` on numpy arrays, so it runs ~5× faster inside the fit and can be unit-tested on its own. The modules the course calls (`workflow.py`, `kinematics.py`, …) are thin wrappers that convert types at the boundary and keep exactly the signatures the notebook and tests expect.

2 The pipeline, step by step (流程)

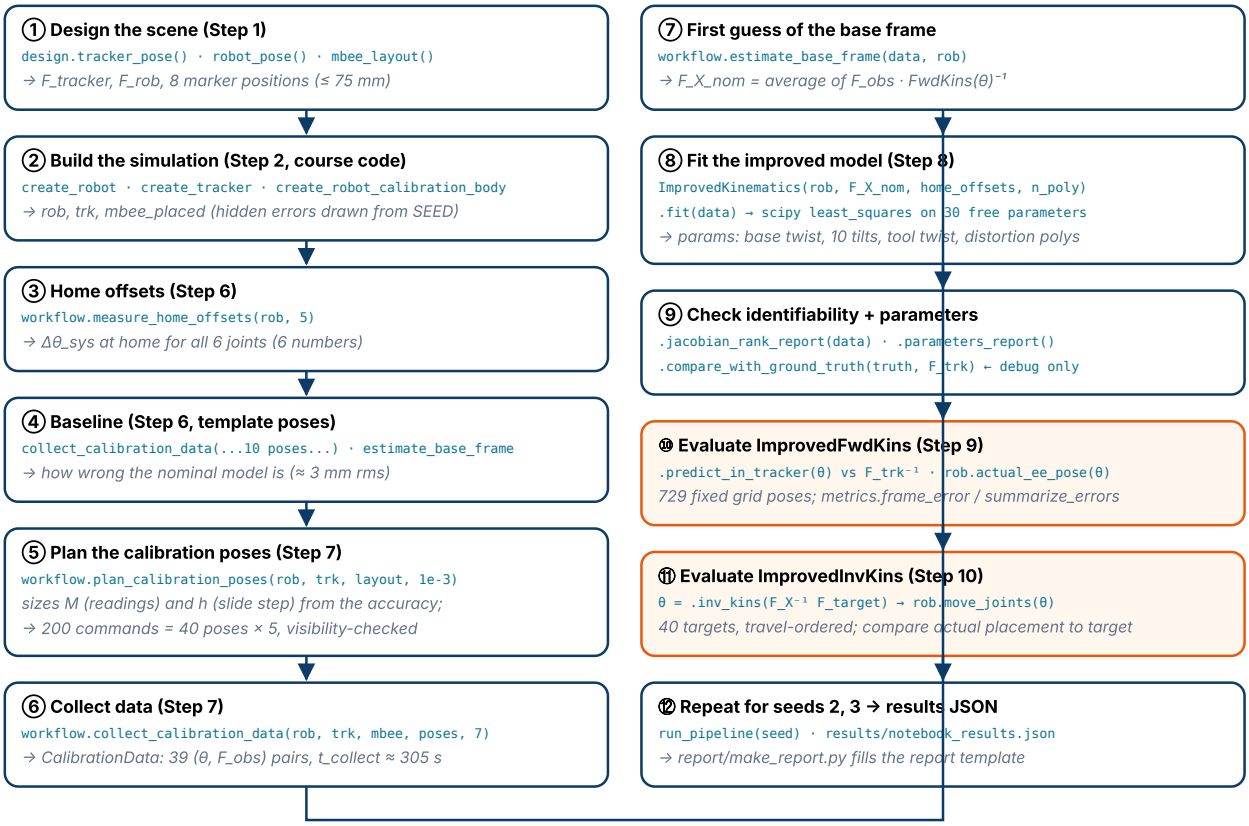


Figure B. The end-to-end flow as the notebook runs it. Function names are the ones you will find in the code.

What flows between the steps (数据流)

Object	Made by	What it holds
`home_offsets (6,)`	`measure_home_offsets`	$\Delta\theta_{sys}$ at home: the encoder values HomeRobot() reports, negated and averaged over 5 homings. Becomes the fixed `joint_offsets` of the model.
`poses, plan_info`	`plan_calibration_poses`	Ordered list of 200 joint vectors (40 distinct poses × 5 re-commands) and a dict: readings per command (7), slide step (75 mm), wrist stops (±1, ±2 rad), predicted time, expected noise.
`CalibrationData`	`collect_calibration_data`	`thetas[j]` encoder values, `frames[j]` the body's measured pose in the tracker frame (one per distinct pose, 35 readings averaged), `readings[j]`, `skipped`, `t_collect`. `as_arrays()` gives numpy (N,6) and (N,4,4) for the core.
`F_X_nom (Frame)`	`estimate_base_frame`	Robot base in tracker coordinates, from the nominal model: a few mm off, used only as the starting point of the fit.
`improved.params (31,)`	`ImprovedKinematics.fit`	The calibrated parameter vector (layout in Part 4). `fit_info` holds the residual statistics.

`results dict → JSON`	notebook	Everything the report needs: per-seed error summaries, times, fit info, rank report, parameters vs truth.

3 Module by module (逐文件说明)

design.py — the Step-1 choices

Pure constants and four tiny functions, kept in code so the notebook and the tests use the same numbers.

Function / constant	What it does
`ENVELOPE_CENTRE, ENVELOPE_HALF`	The marker envelope (标记包络): a cube of half-size 475 mm centred at (800, 0, 300) mm that contains every marker of every reachable pose (wrist reach + 100 mm tool + 75 mm markers).
`tracker_pose()`	Camera 1625 mm above the envelope centre looking straight down (camera +z = world −z). Returns a `Frame`. Gives ≥ 87 mm lateral and 100 mm depth margin, more than the 83 mm worst-case shift of the real base.
`robot_pose()`	`Frame.eye()`: base at the workspace origin (the real base pose is estimated anyway).
`mbee_layout(radius=74.99)`	8 markers at the corners of a cube: the largest allowed spread; orientation noise per reading $0.1/\sqrt{30000} = 5.8 \times 10^{-4}$ rad.
`envelope_corners(F_rob)`	The 8 corners in the workspace; used by the visibility test and the notebook's margin check.

metrics.py — how errors are measured

Function	What it does
`frame_error(F_a, F_b) → (angle, dist)`	$\Delta\mathbf{F} = \mathbf{F}_a^{-1}\mathbf{F}_b$; returns the rotation angle of $\Delta\mathbf{F}.R$ (via the log map, exact for large angles) and $\|\Delta\mathbf{F}.p\|$. Order is (rad, mm), as the template requires.
`summarize_errors(list) → dict`	`n, rms_angle, max_angle, rms_pos, max_pos` plus mean, 95th percentile and `frac_within_target` (share of samples inside 0.2 mm and 1 mrad).
`format_summary(dict) → str`	One readable line for prints.

registration.py — point clouds and frames

Function	What it does
`register_points(a, b) → Frame`	Our own Arun / SVD registration (点云配准): the rigid frame with $\mathbf{F}a_i \approx b_i$. Raises `ValueError` for < 3 points, mismatched lengths or collinear points; a reflection from the SVD is corrected. Wraps `_core.register_points`.
`average_frames(frames) → Frame`	Mean position; rotation = SVD projection of the mean rotation matrix onto a proper rotation (the chordal mean).
`refine_marker_body(trk, placed, n)`	Optional (not needed for the exact calibration body): reads a user body *n* times, maps each reading back into the body frame with `register_points`, averages, and pushes the result into the body with `update_marker_body`.

workflow.py — moving the robot and reading the tracker

Function / class	What it does
`CalibrationData`	Dataclass holding the samples (see Part 2). `as_arrays()` converts to numpy for the core.

`merge_data(*datasets)`	Concatenates data sets (we merge the Step-6 baseline into the fit); sums the time spent.
`measure_home_offsets(rob, n_homings=5)`	Calls `rob.home_robot()` *n* times and returns −mean(encoders): $\Delta\theta_{sys}$ at home. Repeats cost only t_{settle} each.
`pose_is_visible(rob, trk, θ, layout, margin)`	Nominal FK → marker positions in the workspace → `trk.is_visible`, also for every marker shifted ±margin along each axis (covers the real base's placement error).
`predicted_collection_time(rob, poses, readings)`	Sums `rob.motion_time` between consecutive commands plus readings × 0.03 s. Lets the plan report its cost before spending it.
`order_poses_by_motion_time(rob, poses)`	Greedy nearest-neighbour ordering by motion time. Used for the IK test targets (halves the testing time).
`plan_calibration_poses(rob, trk, layout, desired_accuracy, **kw)`	The design heart of the workflow (details below). Returns the list of commands; with `return_info=True` also the sizing dict.
`collect_calibration_data(rob, trk, mbee, thetas, repeats)`	For each command: `rob.move_joints`, then `repeats` × `sample_marker_bodies`; consecutive identical commands are merged into one sample; raw marker positions are averaged and registered once with our `register_points`. Records the encoder values, the frame, the number of readings and the simulated time.
`estimate_base_frame(data, kins)`	$\mathbf{F}_{X,j} = \mathbf{F}_{obs,j} \cdot \mathrm{fwd_kins}(\theta_j)^{-1}$ for every sample, then `average_frames`. Exact for perfect data; a first guess with the nominal model.

INSIDE PLAN_CALIBRATION_POSES

1. **Readings per pose** from the layout: $\sigma_1 = \sigma_b/\sqrt{\lambda_{min}} = 5.8 \times 10^{-4}$ rad per reading; with a budget fraction $f = 0.1$ of the 1 mrad target, $M = \lceil(\sigma_1/(f \cdot acc))^2\rceil = 34$, split over 5 re-commands → 7 readings each (35 total).
2. **Slide step** from the slope bound: $h \le 2f \cdot 0.2/5 \times 10^{-4} = 80$ mm → 75 mm so that it divides 600 mm (9 stops per slide).
3. **Wrist stops**: $\pm w/2, \pm w$ with $w = 2$ rad (the trade-off study showed ±1 rad is marginal, ±2.3 brings nothing more).
4. **Pose list**: home, then each joint's stops with the other joints at their nominal home value; the robot does not return home between joints (the next joint's first stop moves the previous one back in the same command).
5. **Checks**: every pose within joint limits and visible with an 85 mm margin, else `ValueError`. Then the predicted time.

kinematics.py — the calibrated model

`ImprovedKinematics` subclasses the course's `Robot`, so it inherits the limits, timing helpers and `wrist_solutions`. Internally it owns a `_core.ParamModel` and the parameter vector `self.params`.

Method	What it does
`__init__(robot, F_X_init, joint_offsets, n_poly=2, rot_weight=200)`	Builds the model; `n_poly` may be an int or 6 degrees (we use (2,2,2,1,1,1)); `rot_weight` is the mm-per-rad weight in the residual (200 makes 1 mrad count like 0.2 mm).
`initial_params()`	All zeros = the nominal robot at `F_X_init`.
`base_frame(params) / tool_frame / axes / joint_distortion`	The pieces of the model for a parameter vector (default `self.params`).
`fwd_kins(θ, params) → Frame`	**ImprovedFwdKins**: $\theta^* = \theta + \Delta\theta_{sys}(\theta)$, then the chain with calibrated axes and tool frame. Returns $\mathbf{F}_{ee}$ in the robot base frame.
`predict_in_tracker(θ, params)`	`base_frame · fwd_kins`: the pose the tracker should see. Gauge-free; this is what the handout's metric compares.

`inv_kins(F, seed) → θ`	**ImprovedInvKins**, provided by the course: Levenberg–Marquardt iteration on our `fwd_kins`, both wrist branches, nearest in-limit solution.
`free_mask(fix_gauge, fixed)`	Boolean mask of the parameters the fit may change; by default freezes `base_alpha_x` (the unobservable gauge direction).
`residuals(params, data) → (6N,)`	Per sample $[\,w\,\mathrm{Log}(\Delta\mathbf{F}.R);\ \Delta\mathbf{F}.p\,]$ with $\Delta\mathbf{F} = (\mathbf{F}_X\mathbf{F}_{ee}(\theta_j))^{-1}\mathbf{F}_{obs,j}$.
`fit(data, fix_gauge=True) → dict`	Non-linear least squares (scipy, LM, central-difference Jacobian). Sets `self.params`; returns cost, nfev, rms/max residuals.
`jacobian_rank_report(data) → dict`	Singular values of the Jacobian, numerical rank, the weakest directions with parameter names; for all parameters (rank 30/31: the gauge) and for the free ones (30/30).
`parameters_report() → dict`	Axes, tilt vectors, angles between axes, offsets, scale factors, polynomial coefficients, tool frame, base frame.
`compare_with_ground_truth(truth, F_tracker)`	Debug/report only: gauge-free comparison with `rob.ground_truth_parameters()` — angles between axes, axis directions after removing the gauge rotation, distortion functions over the travel, tool and base frame errors.

_core.py — the numpy engine

Function / class	What it does
`skew, exp_so3, log_so3, exp_se3, make_frame, inv_frame, frame_error, project_rotation, average_frames, register_points`	se(3) toolbox (李群/李代数工具) on 4×4 arrays, same conventions as the course's `uncertainty_networks.se3` (twist = [α; ε], right perturbation).
`ParamModel(F_X_init, joint_offsets, n_poly, theta_scale)`	The parameterized robot (Part 4): `base_frame, tool_frame, tilt_vectors, axes, joint_distortion, joint_scale, prepare/fwd_prepared/fwd, predict_tracker, residuals, jacobian, fit, rank_report, report`. `prepare()` computes the θ-independent pieces once per parameter vector, which is what makes the fit fast.
`orientation_noise_per_reading(markers, σ_b)`	$\sigma_b/\sqrt{\lambda_{min}}$ of the layout's inertia-like matrix: the worst-axis orientation noise of one registration.
`readings_needed(σ1, target, fraction)`	$\lceil(\sigma_1/(f\cdot target))^2\rceil$.
`slide_step_from_bounds(ω, pos_target, fraction, travel)`	$h \le 2f\cdot target/\omega$, rounded down to divide the travel.

_convert.py, tests, notebook, report scripts

File	What it does
`_convert.py`	`to_np(Frame)`, `to_frame(4×4)`, `pts_to_np(list of vct3/uvct3)`, `np_to_pts`.
`tests/test_student.py`	30 test functions: metrics (5), registration (8), design/visibility (3), workflow (7), kinematics (6), end to end (1). The course's own tests are kept unchanged; the three it marked TODO are implemented.
`notebooks/assignment2_solution.ipynb`	The template with every TODO filled (Figure B). Ends by writing `results/notebook_results.json`.
`report/variant_study.py`	Runs the pipeline with 10 alternative plans on 10 seeds each → `results/variants.json` (the trade-off table).
`report/make_report.py + template.md`	Fills the Markdown template (Jinja2) from the two JSON files, draws two figures, renders PDF (pandoc + xelatex) and DOCX.

`report/make_notebook.py`	How the solution notebook was generated from the course template (edits specific cells). Not needed to run anything.

4 Inside the model and the fit (模型与拟合)

The parameter vector (参数向量)

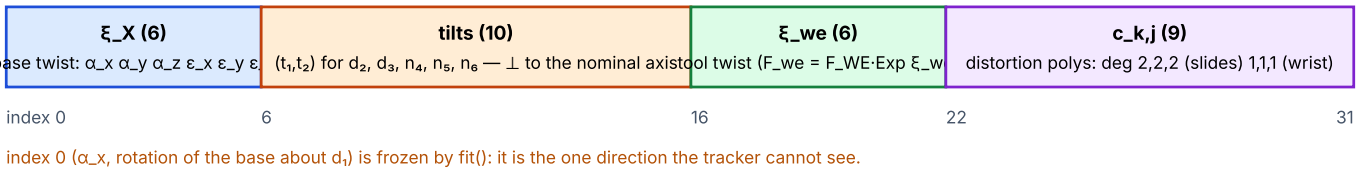
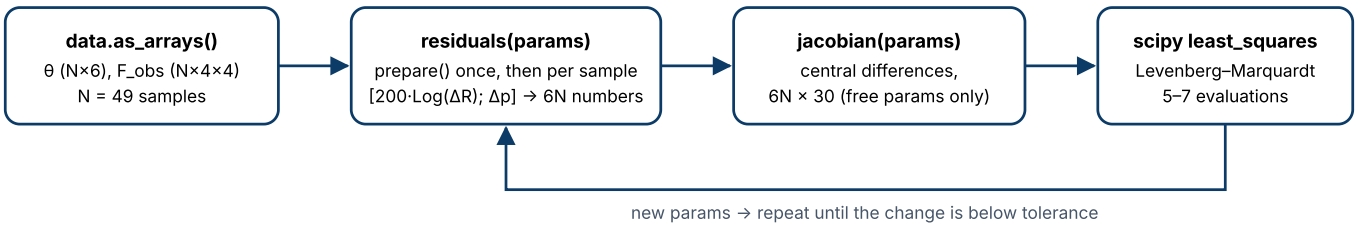


Figure C. Layout of `params`. All zeros = the nominal robot sitting at `F_X_init`. The six joint offsets are not in the vector: they come from homing.

$$\theta_k^* = \theta_k + \underbrace{\text{offset}_k}_{\text{from HomeRobot}} + \sum_{j=1}^{n_k} c_{k,j}\left(\frac{\theta_k}{\theta_k^{max}}\right)^j, \qquad \mathbf{F}_{ee} = \text{Trans}\Big(\textstyle\sum_{k\le 3}\theta_k^*\hat{d}_k + (500,0,0)\Big)\,\mathbf{R}(\hat{n}_4,\theta_4^*)\mathbf{R}(\hat{n}_5,\theta_5^*)\mathbf{R}(\hat{n}_6,\theta_6^*)\,\mathbf{F}_{WE}\,\text{Exp}(\vec{\xi}_{we})$$

$\hat{d}_k = \text{Exp}(t_1\vec{e}_a + t_2\vec{e}_b)\,\vec{d}_k$ with $(\vec{e}_a, \vec{e}_b)$ the two nominal axes perpendicular to $\vec{d}_k$; $\hat{d}_1 = \vec{d}_1$ exactly. The prediction compared with the tracker is $\mathbf{F}_X\mathbf{F}_{ee}$ with $\mathbf{F}_X = \mathbf{F}_{X,init}\,\text{Exp}(\vec{\xi}_X)$.

What happens inside fit() (拟合过程)



Output: self.params (31), fit_info = {cost, nfev, rms_pos_residual_mm ≈ 0.03, rms_rot_residual_rad ≈ 4e-4, max_pos_residual_mm ≈ 0.09}
The residual rms is the tracker's own noise + field: the model explains the data as well as the measurements allow.

Figure D. One fit. Because the real robot is within 1–2 % of the nominal one, the starting point (nominal model at F_X_nom) is already close and the solver needs only a handful of steps.

THE ONE THING THE TRACKER CANNOT SEE (GAUGE 规范自由度)

Rotating the base about $\vec{d}_1$ while tilting $\vec{d}_2, \vec{d}_3, \vec{n}_5, \vec{n}_6$ and the tool frame by the opposite angle gives identical tracker observations. `jacobian_rank_report` shows it as the one zero singular value (rank 30/31), and `fit()` freezes `base_alpha_x` to remove it. Predictions in the tracker frame do not depend on this choice; raw axis vectors do, which is why `compare_with_ground_truth` compares angles between axes or removes the gauge first.

5 Which notebook cell calls what (单元格调用表)

Notebook step	Calls	Prints / produces
Step 1 design	`design.tracker_pose, robot_pose, mbee_layout`	F_tracker_approx, F_rob_approx, mbee_local_positions

Step 2 build	`simulation.create_robot / create_tracker / create_robot_calibration_body / add_marker_bodies`	rob, trk, mbee_placed (course code, SEED = 1)
Step 2.1 visibility	`trk.is_visible, design.envelope_corners`	81/81 grid poses visible; envelope visible under ±85 mm shifts
Step 3 tests	`pytest ../tests`	53 passed
Step 4–5 sanity	`metrics.frame_error, registration.register_points`	two sanity prints
Step 6 home + baseline	`workflow.measure_home_offsets, collect_calibration_data, estimate_base_frame, metrics.*`	home offsets; nominal model vs 10 baseline poses (≈ 3 mm rms)
Step 7 plan + collect	`workflow.plan_calibration_poses(return_info=True), collect_calibration_data(repeats=7), merge_data, estimate_base_frame`	200 commands, 39 samples, 305 s; F_X_nom
Step 8 fit	`kinematics.ImprovedKinematics(...).fit, .jacobian_rank_report, .parameters_report, .compare_with_ground_truth, rob.ground_truth_parameters`	fit info; rank 30/31 → 30/30; parameters vs truth; t_calibration ≈ 389 s
Step 9 FK test	`rob.actual_ee_pose (truth), improved.predict_in_tracker, rob.fwd_kins, metrics.*`	729-pose table: improved ≈ 0.12 mm / 0.35 mrad max
Step 10 IK test	`workflow.order_poses_by_motion_time, kins.inv_kins, rob.move_joints, mbee_placed.actual_placement (truth)`	40 targets × (nominal, improved); t_testing ≈ 480 s
Summary	`metrics.summarize_errors`	the results table; time breakdown
Other seeds	`run_pipeline(2), run_pipeline(3)`	same table per seed; `results/notebook_results.json`

How to change things (如何修改)

If you want to...	Change...
move the camera or change the marker body	`design.py` (one place; tests and notebook follow). Re-run the visibility test.
sweep the wrist further / fewer readings / other spacing	kwargs of `plan_calibration_poses`: `wrist_sweep, readings_per_pose, commands_per_pose, slide_step` (defaults in the docstring).
use another distortion model	`n_poly` in the `ImprovedKinematics(...)` call; the parameter names and the rank report update automatically.
weigh rotation differently in the fit	`rot_weight` (mm per rad, default 200).
re-run everything and refresh the report	run the notebook → `python report/variant_study.py` (optional) → `python report/make_report.py --authors "…"`.

READING ORDER IF YOU WANT TO UNDERSTAND THE CODE

`design.py` → `workflow.py` (plan, collect) → `_core.py` (ParamModel: fwd, residuals, fit) → `kinematics.py` (the wrapper) → the notebook → `tests/test_student.py`. The tests double as worked examples of every function.